



ADDIS ABABA
**SCIENCE AND
TECHNOLOGY**
UNIVERSITY
UNIVERSITY FOR INDUSTRY

COLAGE OF ENGINEERING

Department of Software Engineering

Distributed System Assignment

File Sharing System

Group Members

1. Tilksew Misganaw
2. Bereket Kibret
3. Salhadin Desalegn

ID

CEP0107/14
CEP0309/14
CEP0097/14

Submitted to: Dr. Melak Aye

Due Date: May 2, 2026

Table of Contents

1. Introduction	1
2. System Architecture.....	1
3. Server Design (FileServer.java)	1
3.1 TCP Server Behavior	1
3.2 UDP Server Behavior.....	2
3.3 Client Management	2
4. Client Design (FileClientGUI.java)	2
4.1 Connection Phase	2
4.2 Dashboard Functionality.....	3
5. Peer-to-Peer Communication.....	4
6. File Transfer Mechanisms.....	5
6.1 TCP-Based Transfer	5
6.2 UDP-Based Transfer	5
7. System Workflow.....	5
8. Output and Behavior	5
9. Advantages.....	5
10. Limitations	5
11. Conclusion	6
12. Future Enhancements	6
References:.....	7

File Sharing System (Java TCP/UDP with Peer-to-Peer Communication)

1. Introduction

This project presents the design and implementation of a distributed file sharing system developed using Java. The system combines a centralized server with direct client-to-client communication, allowing users to upload, download, and share files efficiently. Unlike purely centralized systems, this hybrid approach reduces server load by enabling peers to exchange files directly after discovery.

The system supports two transport protocols: TCP and UDP. TCP is used where reliability is critical, such as file downloads and structured communication, while UDP is used to demonstrate faster but less reliable transmission. The inclusion of both protocols provides a practical comparison of their behavior in real-world applications.

2. System Architecture

The system follows a hybrid architecture composed of two major components:

- A centralized server responsible for coordination and storage
- Multiple clients with graphical interfaces that interact with the server and with each other

The server acts as a registry and storage node, while clients function both as users and mini-servers. Once clients are registered, they can discover each other and perform peer-to-peer (P2P) file transfers without routing data through the central server.

3. Server Design (FileServer.java)

The server is the backbone of the system. It listens for incoming client connections and processes requests using multithreading to support concurrent users.

When the server starts, it initializes a storage directory where all uploaded files are stored. It also launches two parallel services: a TCP server and a UDP server, both running on the same port.

3.1 TCP Server Behavior

The TCP server is responsible for handling structured and reliable communication. Each incoming connection is assigned to a separate thread from a cached thread pool, ensuring scalability.

The server processes several commands:

- REGISTER: Stores client ID, IP address, and port
- GET_CLIENTS: Returns a list of other connected clients

- DIR: Lists all files stored on the server
- PUT: Receives a file from a client and saves it
- DOWNLOAD: Sends a requested file to a client

During file transfers, the server reads and writes data in buffered chunks, ensuring efficient memory usage. It also logs progress for large files.

3.2 UDP Server Behavior

The UDP server runs concurrently with the TCP server. It listens for datagram packets and reconstructs files without establishing a connection.

The process begins when a client sends a START message containing the file name. The server then creates a file stream and continuously appends incoming packet data to that file.

Unlike TCP, UDP does not guarantee delivery or order, so this implementation prioritizes simplicity over reliability.

3.3 Client Management

The server maintains a thread-safe list of connected clients. Each client is identified by:

- Client ID
- IP address
- Port number

When a client disconnects, it is removed from the list to ensure only active clients are visible.

4. Client Design (FileClientGUI.java)

The client application is built using Java Swing and provides an interactive graphical interface. It allows users to connect to the server, manage files, and communicate with other clients.

The interface is divided into two main sections: a connection screen and a dashboard.

4.1 Connection Phase

At startup, the user provides:

- Server IP address
- Port number
- Client ID
- Preferred protocol (TCP or UDP)

Once connected, the client registers itself with the server and transitions to the dashboard.

FileShare Client - Direct Client-to-Client File Transfer

FileShare System - Direct Client to Client Sharing

☒ TCP (Reliable) ☐ UDP (Efficient)

Client ID: Client-709

Server IP: 127.0.0.1

Port: 5000

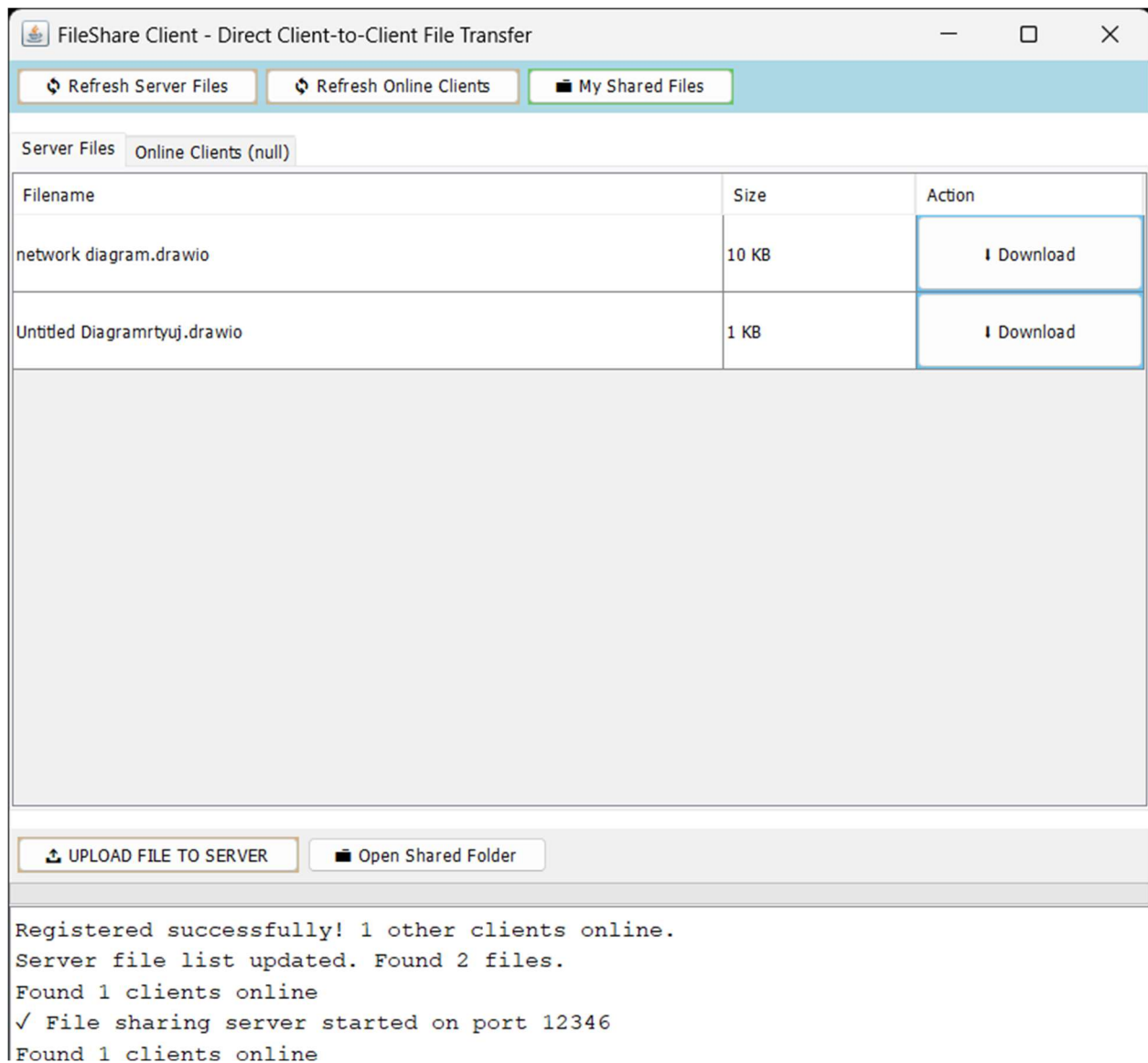
CONNECT TO SERVER

4.2 Dashboard Functionality

The dashboard provides access to all system features. It displays:

- Files available on the server
- List of online clients
- Upload and download controls
- Progress bar and status log

Users can refresh file lists, upload files, and initiate downloads from either the server or other clients.



5. Peer-to-Peer Communication

A key feature of the system is direct file sharing between clients. Each client runs a lightweight server on a local port (12346), allowing it to respond to requests from other clients.

The supported operations include:

- Listing shared files
- Sending files to requesting clients
- Receiving files from other clients

When a user selects another client, a direct socket connection is established, and file transfer occurs without involving the central server.

6. File Transfer Mechanisms

The system implements two different file transfer approaches.

6.1 TCP-Based Transfer

TCP is used for reliable communication. Files are transferred using streams, and progress is tracked continuously. This method ensures that all data arrives correctly and in order.

6.2 UDP-Based Transfer

UDP is used for faster transmission. Files are broken into packets and sent without acknowledgment. While this increases speed, it may result in packet loss or incomplete files.

This dual implementation demonstrates the trade-off between reliability and performance.

7. System Workflow

The typical workflow begins when a client connects to the server and registers itself. After registration, the client retrieves the list of available files and other connected clients.

If the user chooses to upload a file, it is sent to the server using the selected protocol. For downloads, the client requests the file from the server and saves it locally.

For peer-to-peer transfers, the client selects another user, establishes a direct connection, and performs the file transfer independently.

8. Output and Behavior

The system produces output in two forms: console output (server-side) and graphical feedback (client-side).

The server console displays connection logs, commands received, and file transfer progress. This helps in monitoring system activity.

On the client side, users interact through the GUI. Status messages are displayed in a log panel, and a progress bar provides real-time feedback during file transfers. Dialog boxes notify users of success or failure.

9. Advantages

This system offers several benefits. It reduces server load through peer-to-peer communication, supports both TCP and UDP protocols, and provides a user-friendly graphical interface. The use of multithreading ensures that multiple clients can interact with the system simultaneously.

10. Limitations

Despite its functionality, the system has limitations. UDP transfers are not reliable, and there is no built-in mechanism for error recovery. The system also lacks authentication and encryption, making it unsuitable for secure environments. Additionally, it depends on local network configurations, which may affect connectivity.

11. Conclusion

The project successfully demonstrates the implementation of a hybrid file sharing system using Java networking. By integrating centralized control with peer-to-peer communication, it achieves both efficiency and scalability. It also provides a practical comparison between TCP and UDP protocols in file transfer scenarios.

12. Future Enhancements

Future improvements could include adding user authentication, implementing secure communication using encryption, introducing reliable UDP mechanisms, and enhancing the graphical interface. Support for resumable downloads and database integration would further improve system robustness and scalability.

References:

No reference.